02.05.24

OPENCAMPUS.sh

Einführung in Data Science und maschinelles Lernen

VERSIONIERUNG MIT GIT (TEIL 1) UND DATENAUFBEREITUNG

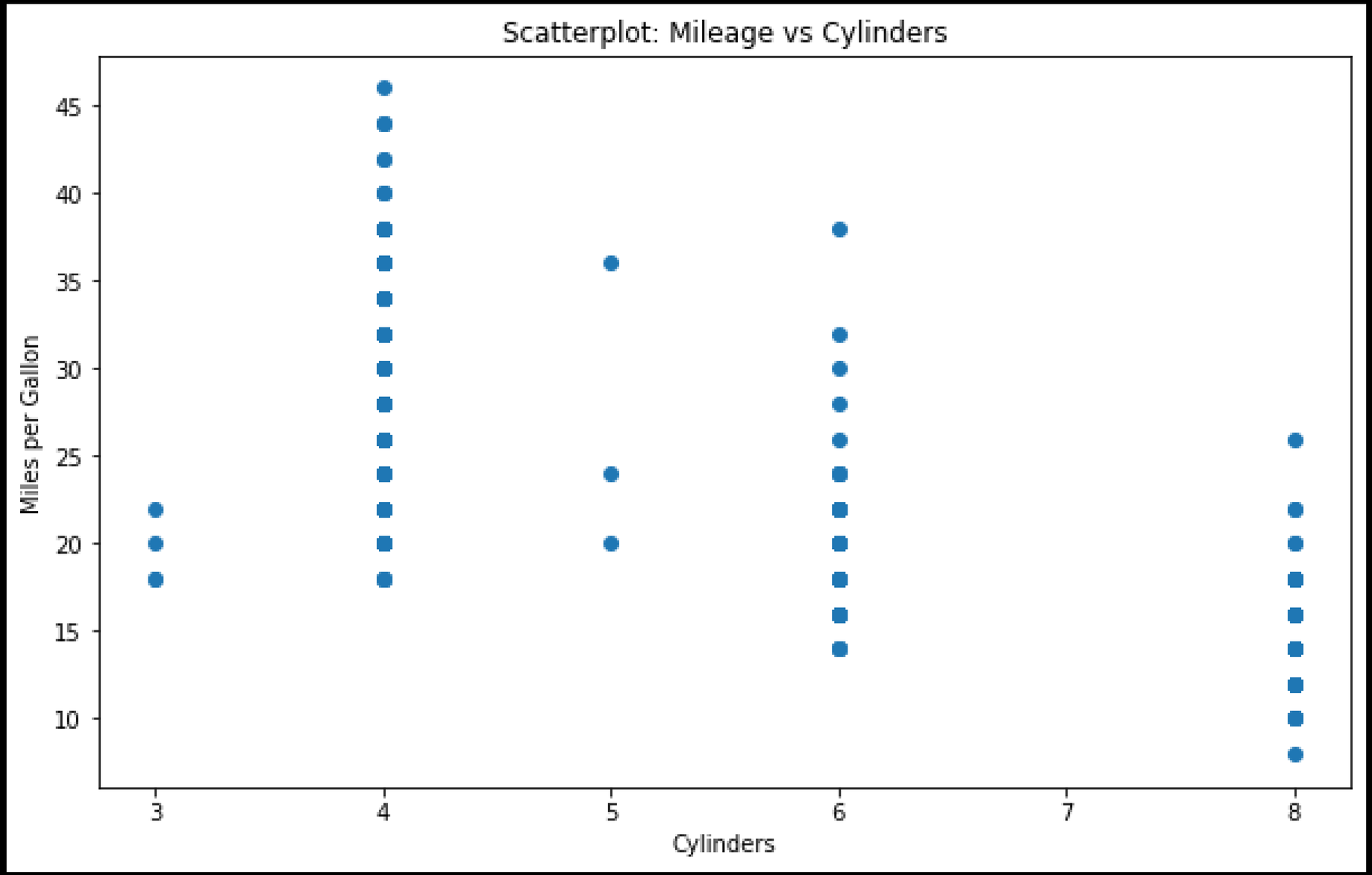
- Zusammenstellung der Teams
- Kennenlernen und Besprechung Übungsaufgaben
- Weitere Beispiele zu Diagrammen
- Statistische Signifikanz
- Einführung in die Versionierung mit git
- Einführung in die Datenaufbereitung

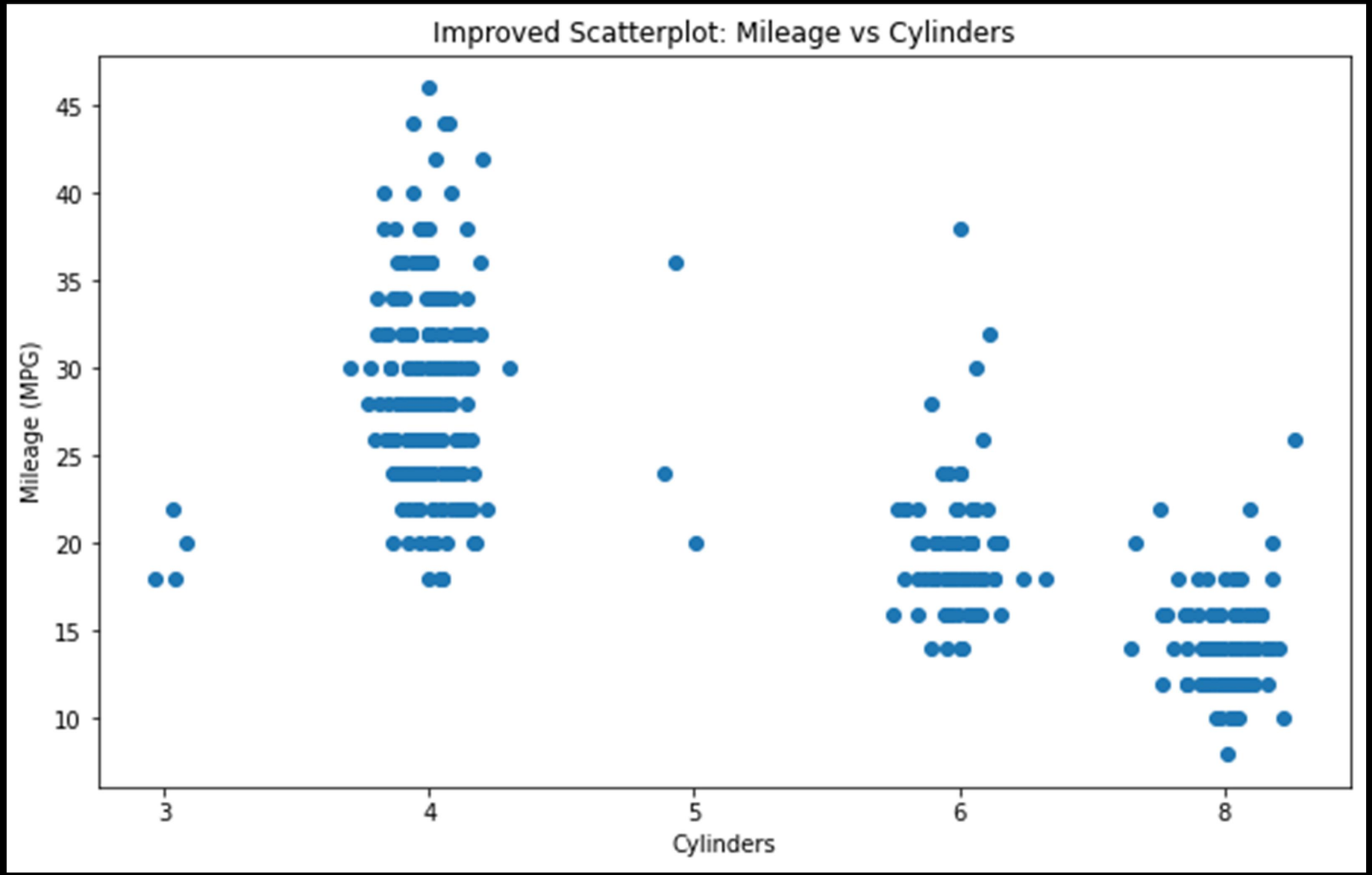
ZUSAMMENSTELLUNG DER TEAMS

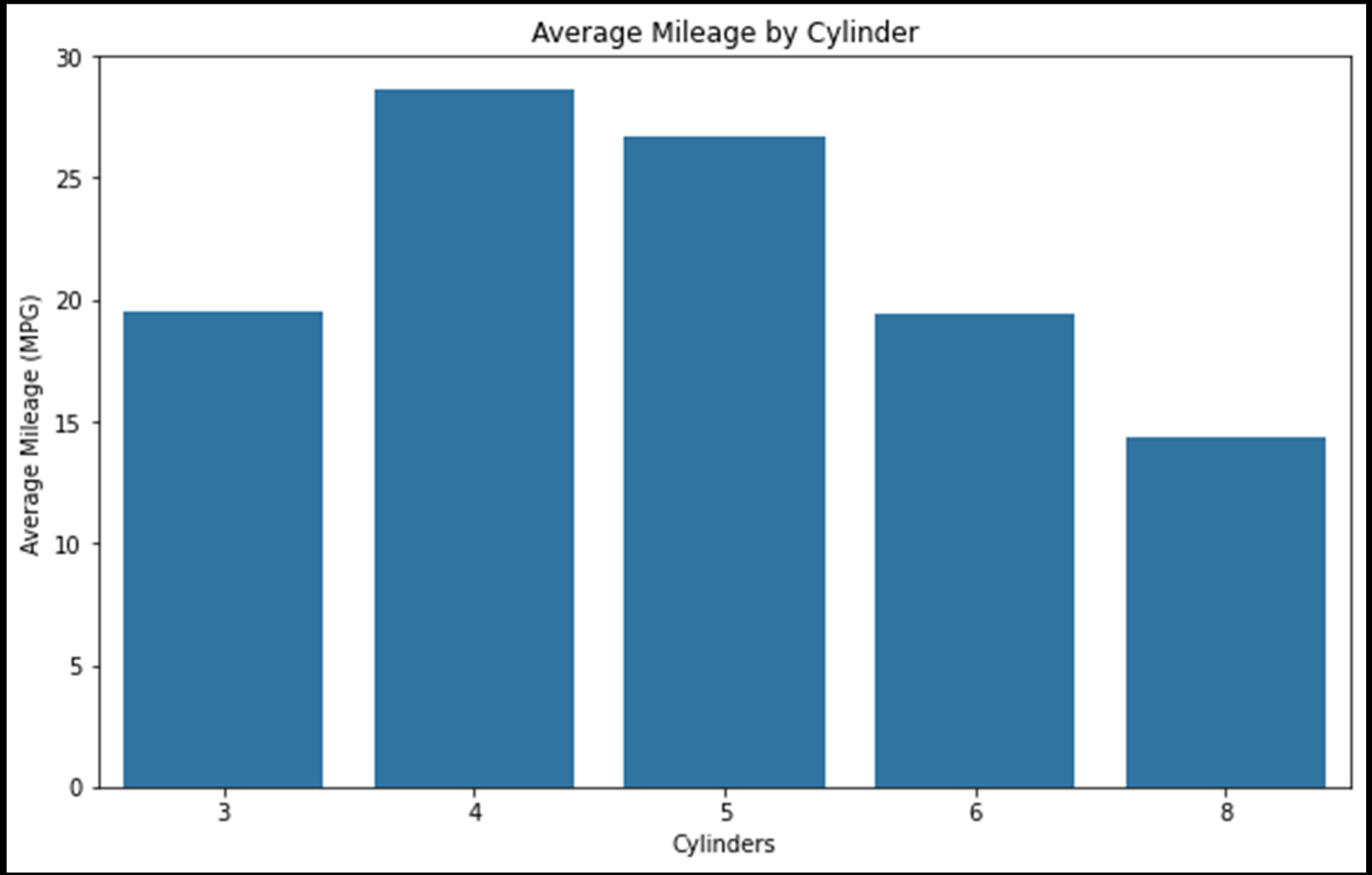
BREAKOUT

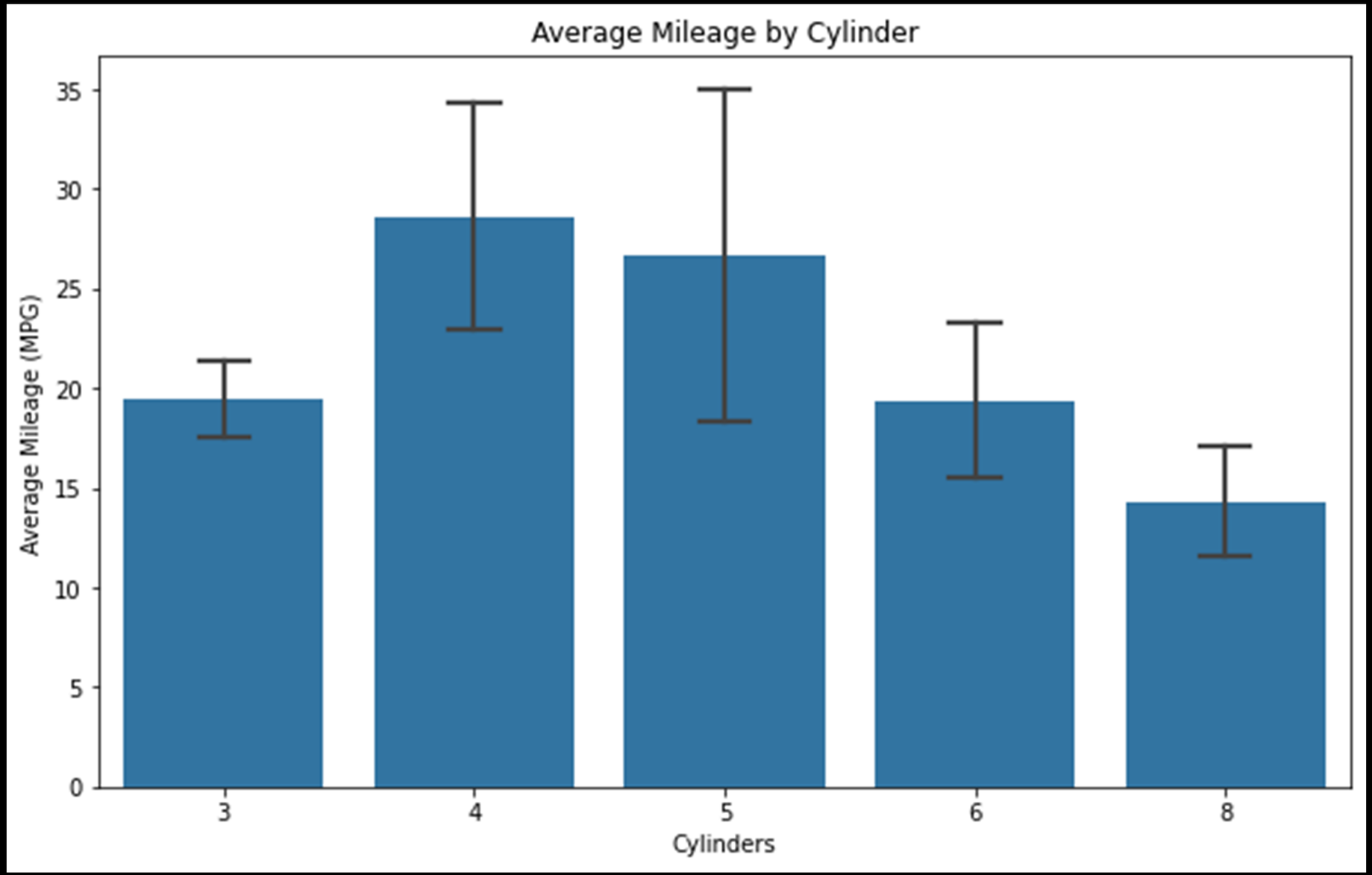
- Vorstellungsrunde
- Anlegen eines Team-Channels im Chat
- Besprechung der Übungsaufgaben

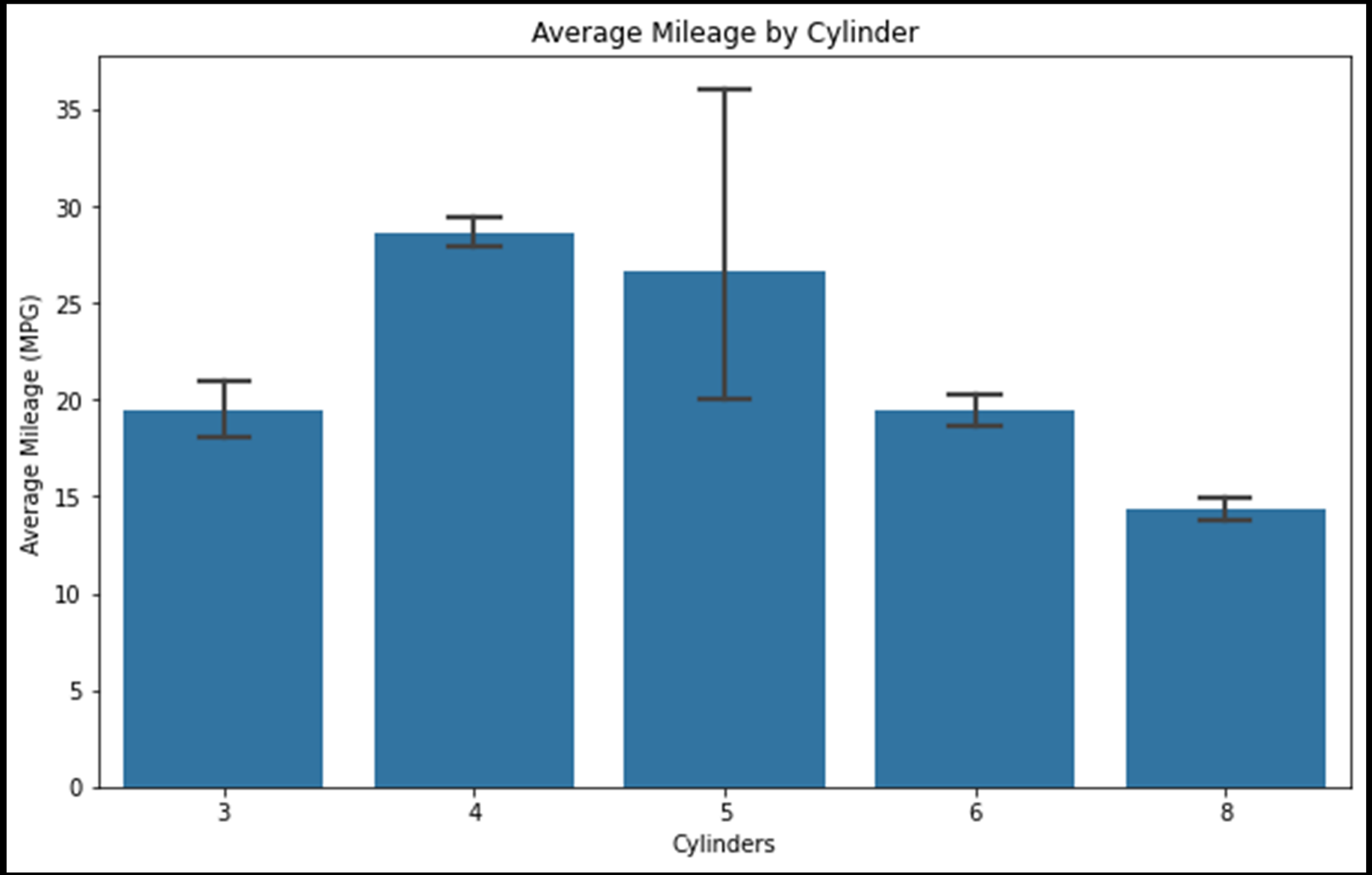
WEITERE BEISPIELE VON VISUALISIERUNGEN


Scatterplot: Mileage vs Cylinders
Miles per Gallon
45
40
35
30
25
20
15
10
3
4
5
6
7
8
Cylinders

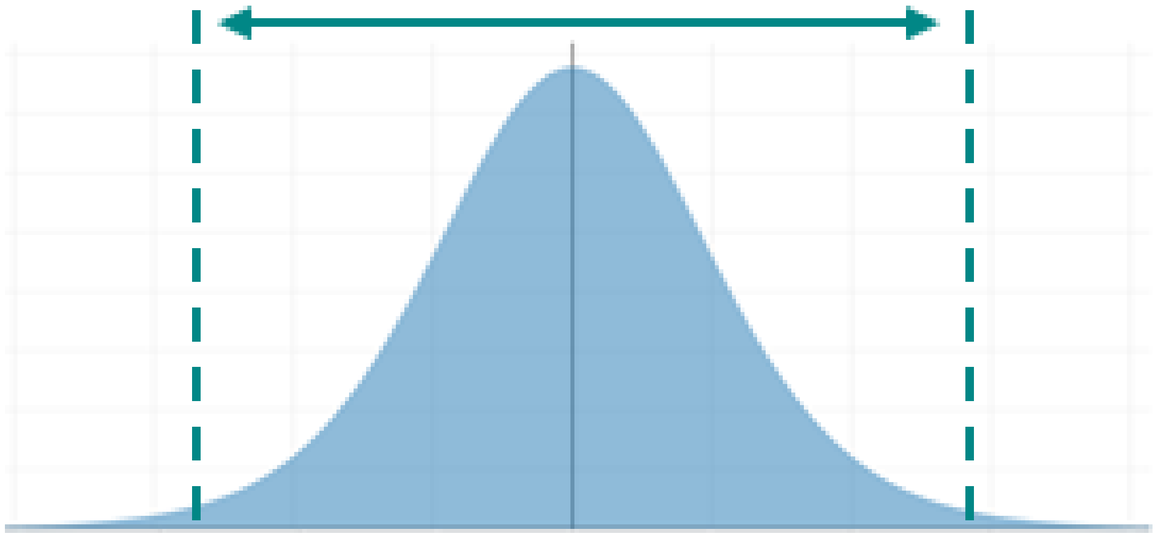

Improved Scatterplot: Mileage vs Cylinders
Mileage (MPG)
45
40
35
30
25
20
15
10
3
4
5
6
8
Cylinders


Average Mileage by Cylinder
Average Mileage (MPG)
30
25
20
15
10
5
0
3
4
5
6
8
Cylinders


Average Mileage by Cylinder
Average Mileage (MPG)
0
5
10
15
20
25
30
35
3
4
5
6
8
Cylinders


Average Mileage by Cylinder
Average Mileage (MPG)
0
5
10
15
20
25
30
35
3
4
5
6
8
Cylinders

Konfidenzintervall



$$x_u = \bar{x} - z \cdot \frac{\sigma}{\sqrt{n}} \qquad x_o = \bar{x} + z \cdot \frac{\sigma}{\sqrt{n}}$$

Mittelwert

Konfidenzniveau

Stichprobengröße

Streuung in der Grundgesamtheit (Standardabweichung)

Standard-fehler

Datatab e.U. (n.d.). Konfidenzintervall. Retrieved November 8, 2021, from Datatab Statistik Rechner website: https://datatab.de/tutorial/konfidenzintervall

STANDARDABWEICHUNG VS. STANDARDFEHLER

Standardabweichung:

- **Information über die Streuung der Beobachtungen zu einer Gruppe**

Standardfehler:

- **Information über den Fehler des Mittelwerts der Beobachtungen zu einer Gruppe, *wenn man diese als repräsentative Stichprobe für eine größere Grundgesamtheit betrachtet.***

SIGNIFIKANZ-TESTS (1)

- **Wie weiß man, ob ein Mittelwert „statistisch signifikant“ oberhalb eines definierten Werts liegt?**
  - **Man weiß mit (in der Regel) 95%er Sicherheit , dass der Mittelwert größer als x ist.**
  - **Das 95%-Konfidenzintervall liegt unterhalb von x.**

SIGNIFIKANZ-TESTS (2)

- Wie weiß man, dass sich zwei Mittelwerte „statistisch signifikant“ voneinander unterscheiden?
  - Man weiß mit (in der Regel) 95%er Sicherheit , dass die Mittelwerte sich unterscheiden.
  - Die 95%-Konfidenzintervalle beider Mittelwerte überschneiden sich nicht.
  - Die Differenz der Mittelwerte ist mit 95%er Sicherheit von 0 verschieden.

T-TEST

```
> t.test(july$Temp, may$Temp)

        Welch Two Sample t-test

data:  july$Temp and may$Temp
t = 12.616, df = 50.552, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 15.43351 21.27617
sample estimates:
mean of x mean of y
 83.90323  65.54839
```

SIGNIFIKANZ VERSUS RELEVANZ

- **Eigenschaften können sich statistisch signifikant unterscheiden, aber der Unterschied kann ohne praktische Relevanz (ohne „Effekt“) sein.**

Betrachtung der Effektstärke nach Cohen:
(Größe des Mittelwertsunterschieds im Vergleich zur Standardabweichung)

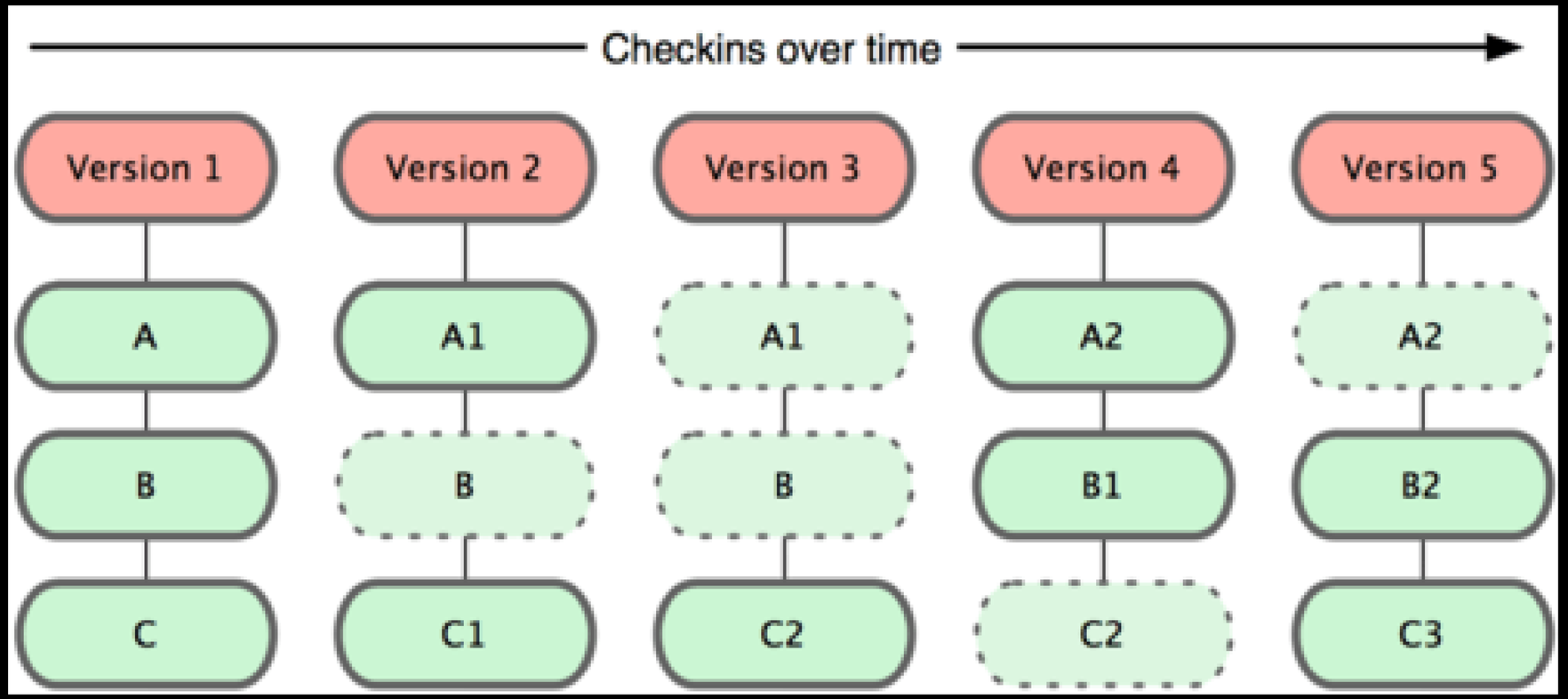
$$d = \frac{\mu_1 - \mu_2}{\sigma}$$

vereinfachter Fall gleicher Stichprobengröße und Standardabweichung

$\lvert d \rvert > 0{,}2$	**kleiner Effekt**
$\lvert d \rvert > 0{,}5$	**mittlerer Effekt**
$\lvert d \rvert > 0{,}8$	**großer Effekt**

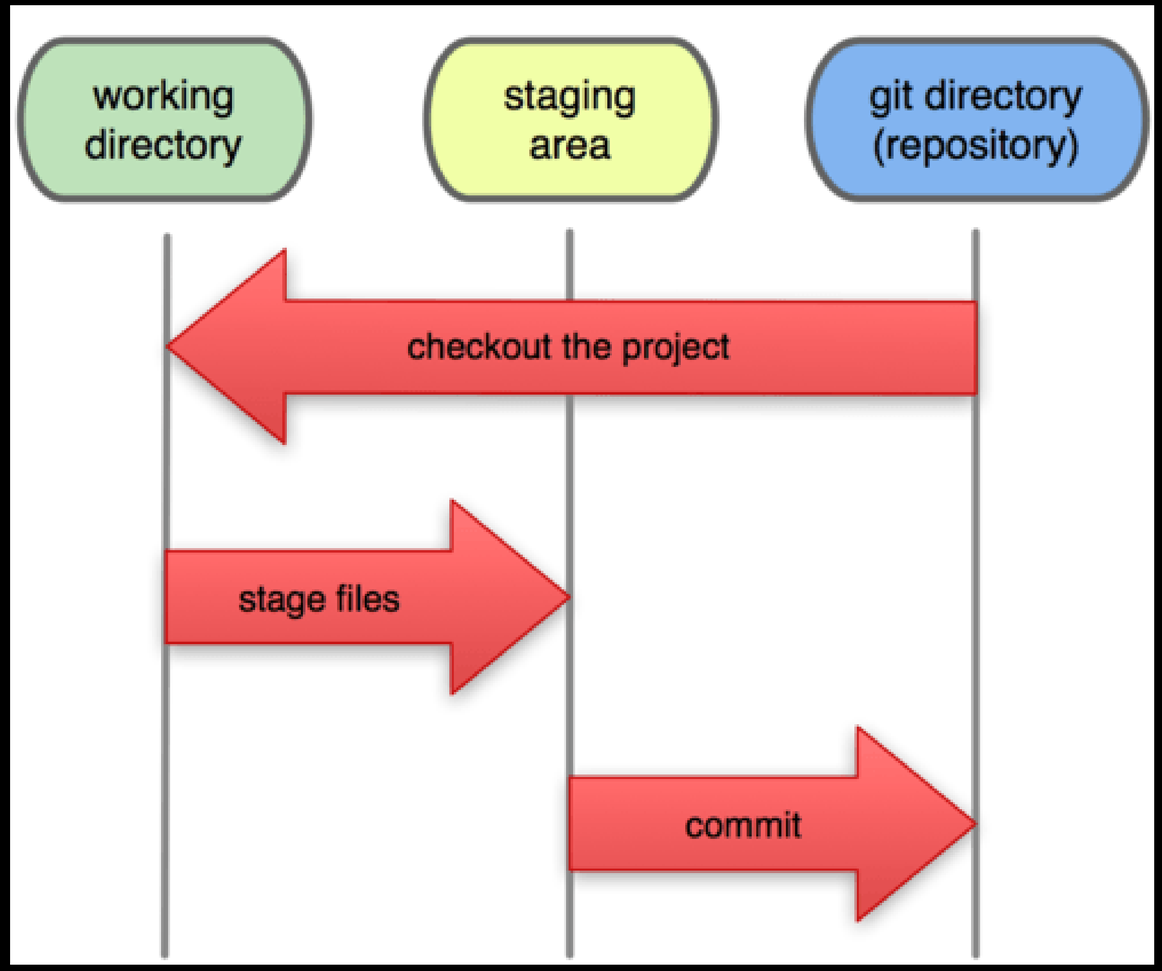
VERSIONIERUNG MIT GIT

LOKALE VERSIONIERUNG MIT GIT



- Alle Versionen werden in einem speziell markierten Verzeichnis, einem „Repository“, abgelegt.
- Jede Version enthält alle Dateien des Projektes, intern werden jedoch nur die eigentlichen Änderungen jeder Version gespeichert.

LOKALE VERSIONIERUNG MIT GIT



Eine Datei kann drei mögliche Zustände haben:

- **modified („geändert“)**
- **staged („vorgemerkt“) und**
- **committed („versioniert“).**

NUTZUNG IN VS CODE

KONFIGURATION VON GIT

Vor der erstmaligen Verwendung von Git, muss (einmalig) definiert werden, in wessen Namen die Repositories des installierten Git verwaltet werden.

- **Führt im „Terminal“ die folgenden zwei Zeilen mit Eurem GitHub-Benutzernamen und Eurer E-Mail-Adresse aus (falls nicht bereits gesetzt):**

```
git config --global user.name "your_username“
git config --global user.email your_email@example.com
```

- **Zur Kontrolle könnte ihr folgenden Befehl eingeben:**

```
git config --list
```

BREAKOUT

1) „Staged“ eine oder mehrere Dateien, die Ihr versionieren wollt und „committed“ sie dann.

2) Führt ein „Commit“ aus, um eine neue Projektversion inklusive der neuen, „gestagten“ Dateien anzulegen.

3) Schaut Euch die History Eures Repositories an.

VERSIONIERUNG MIT GIT

Teil 1: Lokale Versionierung

Teil 2: Synchronisation der lokalen Versionierung mit einer Remote-Versionierung und Arbeiten im Team

DATENAUFBEREITUNG FÜR DAS MASCHINELLE LERNEN

- **Data Collection**
  Sammeln der Rohdaten aus verschiedenen Quellen wie Datenbanken, Dateien, APIs oder Web-Scraping.
- **Cleaning**
  Umgang mit fehlenden Daten, Entfernen von Duplikaten und das Korrigieren von Fehlern.
- **Data Exploration / Analysis**
  Verstehen der Verteilungen von Variablen, Erkennen von Ausreißern und Zusammenhängen., etwa durch statistische Kennzahlen, Korrelationsmatrizen oder Visualisierungswerkzeuge.
- **Feature-Engineering**
  Umwandlung von Rohdaten in Merkmale, die das zugrundeliegende Problem für die Modelle besser darstellen, etwa durch Erstellen neuer Variablen aus vorhandenen Daten und *Kodieren von kategorialen Variablen*.
- **Data Splitting**
  Aufteilung der Daten in Trainings-, Validierungs- und Testdatensätze, um sicherzustellen, dass das Modell gut auf neue, unbekannte Daten generalisieren kann.

JOIN-METHODEN FÜR PANDAS DATAFRAMES

- **Inner Join**
  Verwendet nur die Keys, die in beiden DataFrames vorhanden sind.
  `df1.merge(df2, on='key', how='inner')`

- **Outer Join**
  Verwendet alle Keys aus beiden DataFrames, füllt fehlende Werte mit NaN.
  `df1.merge(df2, on='key', how='outer')`

- **Left Join (and Right Join correspondingly)**
  Verwendet alle Keys aus dem linken DataFrame und fügt passende Einträge aus dem rechten DataFrame hinzu.
  `df1.merge(df2, on='key', how='left’)`

ERSTE DATENINSPEKTION

- **df.head(), df.tail():**
  Zeigt die ersten/letzten Zeilen.
- **df.sample():**
  Zeigt eine zufällige Auswahl von Zeilen.
- **df.shape:**
  Gibt die Dimension des DataFrames zurück.
- **df.info():**
  Zusammenfassung des DataFrames, inkl. Datentypen.

DESKRIPTIVE STATISTIKEN

- **df.describe():**
  Statistische Zusammenfassung der numerischen Spalten.

- **df.isnull():**
  Überprüft auf NaN-Werte.

VISUALISIERUNGEN

- Scatterplots
- Balkendiagramme
- Histogramme

LERNMATERIAL

- Absolviert diesen Kurs bei datacamp zum Zusammenführen von Dataframes.

- Schaut dieses Video zu Git in VS Code.

- Für eine Einführung in die Möglichkeiten von Regular Expressions, schaut Euch dieses Video (11 Minuten) an.

AUFGABEN

- **Importiert die Daten** `umsatzdaten_gekuerzt.csv, kiwo.csv` **und** `wetter.csv` **und führt sie in einem Pandas Dataframe zusammen.**

- **Trefft Euch im Team und teilt auf, wer für welche Variablen deskriptive Statistiken und Visualisierungen erstellt.**

- **Erstellt deskriptive Statistiken und Visualisierungen zu den jeweiligen Variablen.**